



DIPARTIMENTO DI ELETTRONICA,
INFORMAZIONE E BIOINGEGNERIA

Politecnico di Milano

Machine Learning (Code: 097683)

February 11, 2026

Surname:

Name:

Student ID:

Row:

Column:

Time: 2 hours

Maximum Marks: 33

- The following exam is composed of **8 exercises** (one per page). The first page needs to be filled with your **name, surname and student ID**. The following pages should be used **only in the large squares** present on each page. Any solution provided either outside these spaces or **without a motivation** will not be considered for the final mark.
- During this exam you are **not allowed to use electronic devices**, such as laptops, smartphones, smart watches, tablets and/or similar. Only non-programmable calculators are allowed. Any other device must be turned off and stored away for the entire duration of the exam.
- This is a closed-book/closed-notes exam. You are not allowed to bring with you any kind of note, book, written scheme, and/or similar. You are also not allowed to communicate with other students during the exam.
- The violation of the above-mentioned implies the immediate expulsion of the student from the exam room and a **REJECT** mark for the exam.
- Pencils are not allowed. It is your responsibility to provide a readable solution. We will not be held accountable for accidental partial or total cancellation of the exam. The exam can be written either in **English** or **Italian**.
- You are allowed to withdraw from the exam at any time without any penalty. You are not allowed to keep the text of the exam with you while leaving the room.
- **Three of the points will be given on the basis on how quick you are in solving the exam.** If you finish earlier than 30 min before the end of the exam you will get 3 points, if you finish earlier than 20 min you will get 2 points and if you finish earlier than 10 min you will get 1 point (the points cannot be accumulated).
- The box on Page 10 can only be used to complete the Exercises 7, and/or 8.

Ex. 1	Ex. 2	Ex. 3	Ex. 4	Ex. 5	Ex. 6	Ex. 7	Ex. 8	Time	Tot.
/ 7	/ 7	/ 2	/ 2	/ 2	/ 2	/ 4	/ 4	/ 3	/ 33

Exercise 1 (7 marks)

Explain what the Kernel Trick is, what it is used for, and in which ML methods it can be used.

Exercise 2 (7 marks)

Explain the key idea behind Boosting and its main purpose. Describe how training and inference work in Boosting. Finally, give one example of a problem where Boosting is likely to be beneficial and one where it could be harmful or ineffective.

Exercise 3 (7 marks)

Consider the following Python code snippet:

```
1 X = dataset.drop(columns=["label"]).values
2 y = (dataset["label"].values == "spam")
3
4 Xtr, Xtest, ytr, ytest = train_test_split(X, y, test_size=0.3)
5
6 best_acc = -np.inf
7 best_clf = None
8
9 for C in [0.1, 1.0, 10.0]:
10     clf = SVC(C=C, kernel="rbf", gamma=0.5)
11     clf.fit(Xtr, ytr)
12     acc = accuracy_score(ytr, clf.predict(Xtr))
13     if acc > best_acc:
14         best_acc = acc
15         best_clf = clf
16
17 print(f"Chosen C: {best_clf.C}, training accuracy: {best_acc:.3f}")
18 print(f"Test accuracy: {accuracy_score(ytest, best_clf.predict(Xtest)):.3f}")
```

1. Describe the purpose of the code snippet and explain what it does line by line.
2. What is the role of C in the SVM model used in the snippet?
3. Identify *any* methodological issues in the snippet. For each issue, propose a concrete fix (in words or pseudo-code).

1. The snippet trains an RBF-kernel SVM to classify samples as spam/non-spam, selects the hyperparameter C among a few candidates, and then reports the performance on a held-out test set.
 - lines 1--2: extract from the data the input matrix X (all features) and a binary target vector y from the dataset.
 - line 4: splits the data into training and test sets (70%/30%).
 - lines 9--15: for each candidate C , it trains an SVM on (Xtr, ytr) , computes the *training* accuracy, and stores the model with the highest training accuracy.
 - lines 17--18: prints the chosen C , the corresponding training/test accuracy.
2. In (soft-margin) SVM, $C > 0$ controls the strength of regularization: large C penalizes misclassifications more (tighter fit, higher variance risk), while small C increases regularization (wider margin, potentially better generalization).
3. C is selected by maximizing accuracy on training data, which biases selection toward overfitting. It would be better to use a separate validation set or cross-validation on the training data to select C . Also using accuracy for assessment can be misleading under class imbalance (common in spam detection). A more robust approach would be to use metrics like precision, recall, F_1 -score, etc.

Exercise 4 (2 marks)

Indicate whether the following statements about Linear Regression and regularization are true or false. Motivate your answers.

1. If the design matrix Φ is singular, Ridge Regression (with $\lambda > 0$) and Ordinary Least Squares (OLS) yield the same solution.
2. For unregularized Ordinary Least Squares, adding additional features to the model cannot increase the training mean squared error.
3. When outputs are corrupted by noise, adding regularization can reduce the MSE on the test set compared to Ordinary Least Squares.
4. Lasso admits a solution that can be computed by inverting $(\Phi^\top \Phi + \lambda I)$.

1. **False.** If Φ is singular, OLS may have infinite optima. The Ridge solution is unique and well-defined for $\lambda > 0$, but it generally differs from any OLS solution due to the regularization term.
2. **True.** In OLS the training objective is the residual sum of squares; adding features enlarges the hypothesis space, so the minimum over a larger set cannot be higher. Equivalently, the residual after projecting onto a higher-dimensional column space cannot increase, hence training MSE is non-increasing.
3. **True.** With noisy outputs, OLS can have high variance; adding regularization introduces bias but can reduce variance enough to lower the expected test MSE (bias–variance trade-off).
4. **False.** The expression $(\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top y$ is the closed-form *Ridge* solution, not Lasso.

Exercise 5 (2 marks)

Indicate whether the following statements about RL methods are true or false. Motivate your answers.

1. In TD(0) policy evaluation, the update target for $V(S_t)$ uses the one-step bootstrap.
2. In Monte Carlo (MC) policy evaluation, MC first-visit and MC every-visit converge to the same value function V^π (given sufficiently many visits to each state).
3. In Q-learning, the update of $Q(S_t, A_t)$ does not require sampling the next action A_{t+1} in order to compute the TD target.
4. SARSA can be applied to non-Markovian tasks, since it does not rely on the Markov property.

1. **True.** TD(0) updates $V(S_t)$ toward the one-step TD target

$$R_{t+1} + \gamma V(S_{t+1}),$$

which bootstraps from the current estimate instead of using the full return.

2. **True.** In MC *policy evaluation* for a fixed π , both first-visit and every-visit estimate $\mathbb{E}[G_t \mid S_t = s]$; with sufficiently many visits, their sample averages converge to the same V^π .
3. **True.** Q-learning uses the target

$$R_{t+1} + \gamma \max_a Q(S_{t+1}, a),$$

so it does not need to sample the specific next action A_{t+1} (only observe S_{t+1} and R_{t+1}).

4. **False.** Standard SARSA assumes an MDP: the TD target uses $Q(S_{t+1}, A_{t+1})$ and is justified by the Markov property; in non-Markov tasks the state is not sufficient and learning can be inconsistent unless the state is augmented (e.g., with history).

Exercise 6 (2 marks)

You are an ML consultant working with a national park authority that wants to improve both visitor experience and environmental protection.

1. The authority wants to forecast how crowded each area of the park will be over the next hour, based on historical data and contextual information, to plan staff allocation and provide timely recommendations to visitors.
2. During peak days, the park can apply different access rules (e.g., reservation requirements, time-slot limits, dynamic entry caps) to balance congestion and ecological impact. The authority wants an adaptive system that, throughout the day, chooses which rule to apply to keep overcrowding low while maximizing visitor satisfaction.

Model the two problems above as ML problems (e.g., classification, regression, RL, etc.), specifying **the input features or the state representation, the target or the action space, and the loss/reward function**. Also, propose **suitable methods/algorithms** for each task.

1. Crowding forecasting (Supervised learning / Regression).

- **Input features:** for each area i at time t , a feature vector $x_t^{(i)}$ including contextual descriptors (e.g. time-of-day, day-of-week/holiday, weather indicators, special-event flags), operational signals (e.g. park entry counts, parking occupancy), and recent sensor summaries (e.g. current crowd index).
- **Target:** the crowding level one hour ahead, $y_{t+1}^{(i)} \in \mathbb{R}$ (e.g. expected number of visitors or a density index).
- **Loss:** typical regression loss is RSS.
- **Method:** Ridge or Lasso regression to handle correlated features and improve generalization; alternatively KNN regression for local similarity-based predictions, or Gaussian Processes to obtain uncertainty estimates together with $\hat{y}$.

2. Adaptive access management (Reinforcement Learning / Control).

- **State:** s_t summarizing the park situation at decision time t (e.g. current/predicted crowding per area, current access rule, time/calendar context, recent inflow indicators, and proxy signals for satisfaction and environmental stress).
- **Actions:** select an access rule a_t from a discrete set (e.g. enable/disable reservations, increase/decrease an entry cap level, increase/decrease time-slot granularity).
- **Reward:** a single scalar that combines the main goals, e.g.

$$r_t = -\alpha \text{Overcrowding}(s_t) - \beta \text{EcoStress}(s_t) + \gamma \text{Satisfaction}(s_t) - \eta \mathbb{I}[a_t \neq a_{t-1}],$$

where the last term discourages frequent rule changes.

- **Method:** a model-free RL algorithm such as tabular Q-learning if the state can be discretized (or methods involving function approximation otherwise).

Exercise 7 (4 marks)

Suppose we have a training set composed of $N = 120$ samples and we use it to train two binary classifiers $\hat{h}_1 \in \mathcal{H}_1$ and $\hat{h}_2 \in \mathcal{H}_2$, where $|\mathcal{H}_1| = e^5$, $|\mathcal{H}_2| = +\infty$, and $\text{VC}(\mathcal{H}_2) = 3$.

Compute an upper bound on the true classification error with confidence $1 - \delta = 1 - e^{-3}$ for the two classifiers in the following two scenarios:

1. The two classifiers $\hat{h}_1$ and $\hat{h}_2$ are consistent learners.
2. The two classifiers $\hat{h}_1$ and $\hat{h}_2$ have training errors $\hat{\mathcal{L}}(\hat{h}_1) = 0.25$ and $\hat{\mathcal{L}}(\hat{h}_2) = 0.15$, respectively.

Hint. Recall the learning theory bounds:

$ \mathcal{H} < +\infty$	$\mathcal{L}(\hat{h}) \leq \frac{\ln \mathcal{H} + \ln\left(\frac{1}{\delta}\right)}{N}$	$\mathcal{L}(\hat{h}) \leq \hat{\mathcal{L}}(\hat{h}) + \sqrt{\frac{\ln \mathcal{H} + \ln\left(\frac{1}{\delta}\right)}{2N}}$
$ \mathcal{H} = +\infty$		$\mathcal{L}(\hat{h}) \leq \hat{\mathcal{L}}(\hat{h}) + \sqrt{\frac{\text{VC}(\mathcal{H}) \ln\left(\frac{2eN}{\text{VC}(\mathcal{H})}\right) + \ln\left(\frac{4}{\delta}\right)}{N}}$

Data: $\delta = e^{-3}$, $\ln |\mathcal{H}_1| = \ln(e^5) = 5$, and $\ln(1/\delta) = 3$.

• **Scenario 1: consistent learning (training error = 0).**

– For $\hat{h}_1 \in \mathcal{H}_1$ (finite $\mathcal{H}_1$ and consistent):

$$\mathcal{L}(\hat{h}_1) \leq \frac{\ln |\mathcal{H}_1| + \ln(1/\delta)}{N} = \frac{5 + 3}{120} = \frac{8}{120} \approx 0.0667.$$

– For $\hat{h}_2 \in \mathcal{H}_2$ (VC bound with $\hat{\mathcal{L}}(\hat{h}_2) = 0$):

$$\mathcal{L}(\hat{h}_2) \leq \sqrt{\frac{\text{VC}(\mathcal{H}_2) \ln\left(\frac{2eN}{\text{VC}(\mathcal{H}_2)}\right) + \ln\left(\frac{4}{\delta}\right)}{N}}.$$

Here $\text{VC}(\mathcal{H}_2) = 3$, so $\frac{2eN}{3} = 80e$ and $\ln(4/\delta) = \ln(4e^3)$, hence

$$\mathcal{L}(\hat{h}_2) \leq \sqrt{\frac{3 \ln(80e) + \ln(4e^3)}{120}} \approx \sqrt{\frac{20.532}{120}} \approx 0.414.$$

• **Scenario 2: nonzero training errors.**

– For $\hat{h}_1$ with $\hat{\mathcal{L}}(\hat{h}_1) = 0.25$:

$$\mathcal{L}(\hat{h}_1) \leq 0.25 + \sqrt{\frac{\ln |\mathcal{H}_1| + \ln(1/\delta)}{2N}} = 0.25 + \sqrt{\frac{8}{240}} \approx 0.25 + 0.183 = 0.433.$$

– For $\hat{h}_2$ with $\hat{\mathcal{L}}(\hat{h}_2) = 0.15$:

$$\mathcal{L}(\hat{h}_2) \leq 0.15 + \sqrt{\frac{3 \ln(80e) + \ln(4e^3)}{120}} \approx 0.15 + 0.414 = 0.564.$$

Exercise 8 (4 marks)

Consider an algorithm for a MAB with binary rewards, choosing the arm $A_t \in \{a_1, a_2\}$ and obtaining the following rewards R_t at each round t over a time horizon of $T = 12$ rounds. The table shows R_t (recall that only the reward for the chosen arm is revealed to the algorithm):

t	1	2	3	4	5	6	7	8	9	10	11	12
Rewards from a_1	1	0		1		0	1			1		0
Rewards from a_2			0		1			1	0		1	

1. Knowing that the expected reward of a_1 is $\mu_1 = 0.65$ and that of a_2 is $\mu_2 = 0.35$, compute the *expected regret* over the time horizon T .
2. What arm would UCB1 play in the next round $t = 13$? Motivate your answer.
3. What can you say about the arm that Thompson Sampling (initialized with a uniform prior at $t = 0$) would play in the next round $t = 13$? Explain briefly in terms of the posterior distributions.

1. The optimal arm is $a^* = a_1$ since $\mu_1 = 0.65 > \mu_2 = 0.35$. From the table, a_1 is played $N_1 = 7$ times (at $t = 1, 2, 4, 6, 7, 10, 12$) and a_2 is played $N_2 = 5$ times (at $t = 3, 5, 8, 9, 11$). Therefore the expected regret over $T = 12$ rounds is $N_2(\mu_1 - \mu_2) = 5 \cdot 0.30 = 1.5$

2. Empirical means from observed rewards are

$$\hat{\mu}_1 = \frac{1 + 0 + 1 + 0 + 1 + 1 + 0}{7} = \frac{4}{7} \quad \hat{\mu}_2 = \frac{0 + 1 + 1 + 0 + 1}{5} = \frac{3}{5}$$

Using UCB1 index $\hat{\mu}_i + \sqrt{\frac{2 \ln(12)}{N_i}}$, we have

$$\text{UCB}_1 = \frac{4}{7} + \sqrt{\frac{2 \ln(12)}{7}} \approx 1.41 \quad \text{UCB}_2 = \frac{3}{5} + \sqrt{\frac{2 \ln(12)}{5}} \approx 1.60,$$

Hence UCB1 would select a_2 at $t = 13$ (larger upper confidence bound).

3. Thompson Sampling at $t = 13$. With a uniform prior Beta(1, 1) and Bernoulli rewards, the posteriors are

$$a_1 : s_1 = 4, f_1 = 3 \Rightarrow \theta_1 \sim \text{Beta}(1 + s_1, 1 + f_1) = \text{Beta}(5, 4),$$

$$a_2 : s_2 = 3, f_2 = 2 \Rightarrow \theta_2 \sim \text{Beta}(4, 3).$$

Thompson Sampling draws one sample from each posterior and plays the arm with the larger draw. Since $\mathbb{E}[\theta_2] = \frac{4}{7} \approx 0.571$ is slightly larger than $\mathbb{E}[\theta_1] = \frac{5}{9} \approx 0.556$ (and θ_2 is based on fewer samples, hence more variable), it is *more likely* to pick a_2 , but the choice is stochastic and a_1 can still be selected with non-zero probability.

